

Brett Bryant

COIN 496 Senior Portfolio Review

April 9th, 2010.

Programs



Bloggernaut (COIN 332 Applied Networking)

Bloggernaut is a website developed with ASP.net that allows users to sign up and make posts in their blog. The languages used were ASP.net (Visual Basic), HTML, CSS and SQL. For ease of use, both the complete source code and the Visual Basic solution file is provided on the CD.

Bloggernaut has a lot of source code, so you'll notice that only the first page of source code and a screenshot for each website page are the only things visible in this binder. The rest of the source code for each page is provided behind the first one in the sleeves.

Grade: A



Supermarket Queue (COIN 325 Java I)

The supermarket Queue is a computer simulation of the congestion that could occur in a line at the supermarket. This java program randomly adds or removes a person from line every 1-4 iterations (or 1-4 simulation minutes). The result is figuring out what the largest line that built up was, and how long the slowest customer moved through the line.

Grade: A



Number to Words (COIN 333 Applied Systems)

This set of programs utilizes a custom made Perl package which can convert any integer within a reasonable range, which is good as long as the integer is less a zillion away from zero, to it's equivalent composition in english words.

Grade: A (overall grade)



Indexed Multi List (COIN 315 Data Structures)

Written in C++, the IndexedMultiList class implements a singly linked list between individual characters that can be added, modified, removed, and searched in the list like a normal linked list with the added benefit that the IndexedMultiList class also has functions available for using the index of the list node to add, modify, remove and search as well.

Grade: B+ (overall grade)

Bloggermault

Login

Register

Welcome to Bloggermault!

Bloggermault is a website which allows it's users a forum to write and distribute any content they would like to post on the Internet. Registering for Bloggermault is easy, get started or login now.

Recent Blog Posts

- 'this is a test post' at *This is a blog* posted on 12/7/2009 7:42:14 PM
- 'test' at *Testing* posted on 12/7/2009 6:12:05 PM
- 'asfa' at *Computer Science* posted on 12/7/2009 5:55:00 PM
- 'asd' at *Computer Science* posted on 12/7/2009 5:52:49 PM
- 'asfasf' at *Computer Science* posted on 12/7/2009 5:51:20 PM
- 'test' at *Computer Science* posted on 12/7/2009 3:38:42 PM

```

' Brett Bryant
'-----
' Program name: Bloggernaut
'
' Created:      November 9th, 2009
' Last Modified: November 16th, 2009
'
' Course:      COIN 325 Java I
' Grade Received: A
'-----
'
' Comments:
'   This code is responsible for showing
'   detailed error messages from the ViewPost or
'   ViewBlog page if they redirect here on failure
'   as well as listing every post on the site in
'   an unordered list for people to choose from.
'
Imports System
Imports System.Data
Imports System.Data.Sql
Imports System.Data.SqlClient

Partial Class _Default
    Inherits System.Web.UI.Page

    Protected Sub Page_Load(ByVal sender As Object, ByVal e As System.EventArgs) Handles Me.Load
        Dim reader As SqlDataReader

        reader = SqlDataSource1.Select(DataSourceSelectArguments.Empty)

        If Request.QueryString("Error") = 1 Then
            [Error].Text = "<p class='error'><strong>Error:</strong> The blog you are
trying to view does not exist!</p>"
        ElseIf Request.QueryString("Error") = 2 Then
            [Error].Text = "<p class='error'><strong>Error:</strong> The blog post you
are trying to view does not exist!</p>"
        End If

        If reader.Read() Then
            RecentPosts.Text = "<ul>"

            Do
                RecentPosts.Text += "<li><a href='ViewPost.aspx?ID=" + reader("ID").
ToString() + "'>" + reader("Title") + "</a> at <em>" + reader("BlogName") + "</em>
posted on " + reader("DatePosted") + "</li>"
            Loop While reader.Read()

            RecentPosts.Text += "</ul>"
        Else
            RecentPosts.Text = "<p>It doesn't look like you've made any posts yet.</p>"
        End If
    End Sub
End Class

```

```

<%@ Page Language="VB" AutoEventWireup="false" CodeFile="Default.aspx.vb" Inherits=
    "_Default" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>Bloggernaut: Homepage</title>
    <style type="text/css" media="screen">
        @import url(_css/design.css);
    </style>
</head>
<body>
    <div id="header">
        <a href="#"></a>
    </div>
    <div id="navigation">
        <ul>
            <li class="first">
                <form id="Form1" runat="server">
                    <asp:loginstatus ID="Loginstatus1" runat="server"></asp:loginstatus>
                </form>
            </li>
            <asp:LoginView ID="LoginView1" runat="server">
                <AnonymousTemplate>
                    <li><a href="Register.aspx">Register</a></li>
                </AnonymousTemplate>
                <LoggedInTemplate>
                    <li><a href="user/Blog.aspx">My Blog</a></li>
                </LoggedInTemplate>
            </asp:LoginView>
        </ul>
    </div>
    <div id="content">
        <asp:SqlDataSource ID="SqlDataSource1" runat="server" datasourcemode="DataReader"
            ConnectionString="<%%$ ConnectionStrings:ConnectionString %>"
            SelectCommand="SELECT [BlogPosts].*,[Blogs].[BlogName] FROM [BlogPosts] LEFT
JOIN [Blogs] ON [BlogPosts].[BlogID] = [Blogs].[ID] ORDER BY [DatePosted] DESC"></asp:
SqlDataSource>
        <div class="section">
            <h1>Welcome to Bloggernaut!</h1>
            <p>
                Bloggernaut is a website which allows it's users a forum to write and
                distribute any content they would like to post on the internet.
                <asp:LoginView ID="LoginView2" runat="server">
                    <AnonymousTemplate>
                        <a href="Register.aspx">Registering</a> for Blaggernauth is easy,
                        <a href="Register.aspx">get started</a> or <a href="Login.aspx">
login now</a>.
                    </AnonymousTemplate>
                    <LoggedInTemplate>
                        <a href="user/Blog.aspx">Go to your blog</a>!
                    </LoggedInTemplate>
                </asp:LoginView>
            </p>
            <asp:Label ID="Error" runat="server" Text=""></asp:Label>
        </div>
        <div class="section">
            <h1>Recent Blog Posts</h1>
            <asp:Label ID="RecentPosts" runat="server" Text=""></asp:Label>
        </div>
    </div>
    <div id="footer">
        Copyright &copy 2009, Brett Bryant, all rights reserved.
    </div>

```

```
</body>  
</html>
```

```
/*
 * Brett Bryant
 * -----
 * Program name:    Bloggernaut
 *
 * Created:         November 9th, 2009
 * Last Modified:   November 16th, 2009
 *
 * Course:          COIN 332 Applied Networking
 * Grade Received:  A
 * -----
 *
 * Comments:
 *     This is the CSS that implements Bloggernaut's
 *     visual design.
 */

body
{
    font-family: Helvetica, Sans-Serif;
    font-size: 62.5%;
    background: #222;
    margin: 0;
    padding: 0;
}

a, a:visited, a:active
{
    color: #446354;
    text-decoration: none;
    border-bottom: 1px dashed #446354;
}

a:hover
{
    border-bottom: 1px solid #446354;
}

h1, h2, h3
{
    border-bottom: 1px solid black;
    margin-bottom: 0;
}

h1
{
    font-size: 2.4em;
}

h2
{
    font-size: 2em;
}

h3
{
    font-size: 1.6em;
}

p.error
{
    color: Red;
    text-decoration: underline;
}
```

```
#header, #navigation
{
    padding: 5px;
    margin: 5px auto 5px auto;
    width: 950px;
    color: #446354;
    font-size: 1.6em;
    clear: both;
}

#header a, #header img
{
    border: 0;
}

#navigation
{
    margin-bottom: 30px;
}

#navigation ul
{
    display: block;
    clear: right;
    font-family: Verdana, sans-serif;
    margin: 0;
    padding: 0;
}

#navigation ul li
{
    list-style-type: none;
    width: 150px;
    border-right: 1px solid #446354;
    text-align: center;
    font-weight: bold;
    float: left;
}

#navigation li.first
{
    border-left: 1px solid #446354;
}

#content
{
    background: #222;
    border: 2px solid #777;
    width: 950px;
    padding: 5px;
    margin: 5px auto;
    font-size: 12px;
    height: 80%;
    overflow: auto;
}

#content .section
{
    padding: 5px 10px 5px 10px;
    background: white;
    margin: 0;
}

#footer
{
    padding: 5px;
```

```
margin: 5px auto;  
width: 950px;  
color: #446354;  
font-style: italic;
```

```
.display-none
```

```
{  
    display: none;  
}
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head id="Head1" runat="server">
    <title>Bloggernaut: Register</title>
    <style type="text/css" media="screen">
        @import url(_css/design.css);
    </style>
</head>
<body>
    <div id="header">
        <a href="Default.aspx"></a>
    </div>
    <div id="navigation">
        <ul>
            <li class="first"><a href="Login.aspx">Login</a></li>
            <li><a href="Register.aspx">Register</a></li>
        </ul>
    </div>
    <div id="content">
        <div class="section">
            <h1>Register for Bloggernaut...</h1>
            <form runat="server">
                <asp:CreateUserWizard ID="CreateUserWizard1" runat="server"
                    ContinueDestinationPageUrl="user/Blog.aspx">
                    <TitleTextStyle CssClass="display-none" />
                    <WizardSteps>
                        <asp:CreateUserWizardStep ID="CreateUserWizardStep1" runat=
"server">
                            </asp:CreateUserWizardStep>
                            <asp:CompleteWizardStep ID="CompleteWizardStep1" runat="server">
                                </asp:CompleteWizardStep>
                        </WizardSteps>
                    </asp:CreateUserWizard>
                </form>
            </div>
        </div>
    <div id="footer">
        Copyright &copy; 2009, Brett Bryant, all rights reserved.
    </div>
</body>
</html>
```



Login Register

Register for Bloggernaut...

User Name:

Password:

Confirm Password:

E-mail:

Security Question:

Security Answer:



http://localhost:60525/BloggerNaut/Login.aspx



Google

BloggerNaut: Login

BloggerNaut

Login

Register

Login to BloggerNaut...

User Name:

Password:

☐ Remember me next time.

Copyright © 2009, BloggerNaut, all rights reserved.

Done

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head id="Head1" runat="server">
  <title>Bloggernaut: Login</title>
  <style type="text/css" media="screen">
    @import url(_css/design.css);
  </style>
</head>
<body>
  <div id="header">
    <a href="Default.aspx"></a>
  </div>
  <div id="navigation">
    <ul>
      <li class="first"><a href="Login.aspx">Login</a></li>
      <li><a href="Register.aspx">Register</a></li>
    </ul>
  </div>
  <div id="content">
    <div class="section">
      <h1>Login to Bloggernaut...</h1>
      <form runat="server">
        <asp:Login ID="Login1" runat="server" TitleText="">
        </asp:Login>
      </form>
    </div>
  </div>
  <div id="footer">
    Copyright &copy 2009, Brett Bryant, all rights reserved.
  </div>
</body>
</html>
```

Blogger.naut

Logout

My Blog

Starting a blog...

What would you like to call your blog?

Create

Copyright © 2009, Brett Egan. All rights reserved.

```
' Brett Bryant
' -----
' Program name: Bloggernaut
'
' Created:      November 9th, 2009
' Last Modified: November 16th, 2009
'
' Course:      COIN 325 Java I
' Grade Received: A
' -----
'
' Comments:
'   This page is responsible for two things:
'       1) Rerouting people already with blogs before the
'           page starts to go to BlogPost.aspx since only
'           only blog is allowed per user.
'
'       2) Creating a blog for users without one.
```

```
Imports System
Imports System.Data
Imports System.Data.Sql
Imports System.Data.SqlClient
```

```
Partial Class user_BlogCreate
    Inherits System.Web.UI.Page
```

```
Protected Sub Page_Load(ByVal sender As Object, ByVal e As System.EventArgs) Handles Me.Load
    Dim reader As SqlDataReader

    SqlDataSource1.SelectParameters("UserID").DefaultValue = Membership.GetUser().
ProviderUserKey.ToString()
    reader = SqlDataSource1.Select(DataSourceSelectArguments.Empty)

    If reader.Read() Then
        Response.Redirect("Blog.aspx")
    End If
End Sub
```

```
Protected Sub Button1_Click(ByVal sender As Object, ByVal e As System.EventArgs) Handles Button1.Click
    SqlDataSource1.InsertParameters("UserID").DefaultValue = Membership.GetUser().
ProviderUserKey.ToString()
    SqlDataSource1.InsertParameters("UserEmail").DefaultValue = Membership.GetUser().
Email
    SqlDataSource1.InsertParameters("BlogName").DefaultValue = BlogName.Text
    SqlDataSource1.Insert()
    Response.Redirect("Blog.aspx")
End Sub
End Class
```

```

<%@ Page Language="VB" AutoEventWireup="false" CodeFile="BlogCreate.aspx.vb" Inherits="
    "user_BlogCreate" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head id="Head1" runat="server">
    <title>Bloggeronaut: Blog: Create</title>
    <style type="text/css" media="screen">
        @import url(../css/design.css);
    </style>
</head>
<body>
    <div id="header">
        <a href="../Default.aspx"></a>
    </div>
    <form id="Form1" runat="server">
    <div id="navigation">
        <ul>
            <li class="first">
                <asp:loginstatus ID="Loginstatus1" runat="server"></asp:loginstatus>
            </li>
            <asp:LoginView ID="LoginView1" runat="server">
                <AnonymousTemplate>
                    <li><a href="../Register.aspx">Register</a></li>
                </AnonymousTemplate>
                <LoggedInTemplate>
                    <li><a href="Blog.aspx">My Blog</a></li>
                </LoggedInTemplate>
            </asp:LoginView>
        </ul>
    </div>
    <div id="content">
        <asp:SqlDataSource ID="SqlDataSource1" runat="server">
            ConnectionString="<%= $ ConnectionStrings:ConnectionString %>"
            SelectCommand="SELECT * FROM [Blogs] WHERE ([UserID] = @UserID)"
            DataSourceMode="DataReader"
            InsertCommand="INSERT INTO Blogs(UserID, userEmail, BlogName, LastUpdated)
VALUES (@UserID, @UserEmail, @BlogName, 0)"
            <SelectParameters>
                <asp:Parameter Name="UserID" Type="String" />
            </SelectParameters>
            <InsertParameters>
                <asp:Parameter Name="UserID" />
                <asp:Parameter Name="UserEmail" />
                <asp:Parameter Name="BlogName" />
            </InsertParameters>
        </asp:SqlDataSource>
        <div class="section">
            <h1>Starting a blog...</h1>
            <p>
                <strong>What would you like to call your blog?</strong><br />
                <asp:TextBox ID="BlogName" runat="server"></asp:TextBox>
                <asp:Button ID="Button1" runat="server" Text="Create" />

                <asp:RequiredFieldValidator ID="RequiredFieldValidator1" runat="server"
                    ControlToValidate="BlogName" ErrorMessage="You must enter a blog name"
                    . "></asp:RequiredFieldValidator>
            </p>
        </div>
    </div>
    <div id="footer">
        Copyright &copy; 2009, Brett Bryant, all rights reserved.
    </div>

```

</body>
</html>

Blogger.naut

Logout

My Blog

Welcome to Your Blog: portfolio !

[Make a new post.](#)

[Permalink to your blog.](#)

Your Recent Blog Posts

It doesn't look like you've made any posts yet.

Copyright © 2004. Blogger.naut. All rights reserved.

Done

```

'
' Brett Bryant
' -----
' Program name: Bloggernaut
'
' Created:      November 9th, 2009
' Last Modified: November 16th, 2009
'
' Course:      COIN 325 Java I
' Grade Received: A
' -----
'
' Comments:
'   This page is responsible for coordinating
'   a user's tasks with their blogs. If you don't
'   have a blog yet, this page will redirect you to
'   create one. Otherwise, you'll see a list of your
'   recent posts and have the ability to create new ones.
'
Imports System
Imports System.Data
Imports System.Data.Sql
Imports System.Data.SqlClient

Partial Class user_Blog
    Inherits System.Web.UI.Page

    Private Sub Page_Load(ByVal sender As Object, ByVal e As System.EventArgs) Handles Me.Load
        Dim UserID As String
        Dim reader As SqlDataReader

        UserID = Membership.GetUser().ProviderUserKey.ToString()

        SqlDataSource1.SelectParameters("UserID").DefaultValue = UserID
        reader = SqlDataSource1.Select(DataSourceSelectArguments.Empty)

        If Not reader.Read() Then
            Response.Redirect("BlogCreate.aspx")
        End If

        Permalink.Text = "<a href='../ViewBlog.aspx?ID=" + reader("ID").ToString() + "'>
        Permalink to your blog.</a>"

        BlogName.Text = reader("BlogName")

        SqlDataSource1.SelectCommand = "SELECT * FROM [BlogPosts] WHERE ([UserID] = @
        UserID) ORDER BY [DatePosted] DESC"
        reader = SqlDataSource1.Select(DataSourceSelectArguments.Empty)

        If reader.Read() Then
            RecentPosts.Text = "<ul>"

            Do
                RecentPosts.Text += "<li><a href='../ViewPost.aspx?ID=" + reader("ID").
                ToString() + "'>" + reader("Title") + "</a> posted on " + reader("DatePosted") + "
                </li>"
            Loop While reader.Read()

            RecentPosts.Text += "</ul>"
        Else
            RecentPosts.Text = "<p>It doesn't look like you've made any posts yet.</p>"
        End If
    End Sub
End Class

```

```

<%@ Page Language="VB" AutoEventWireup="false" CodeFile="Blog.aspx.vb" Inherits=
    "user_Blog" debug="true" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head id="Head1" runat="server">
    <title>Bloggernaut: Blog: Homepage</title>
    <style type="text/css" media="screen">
        @import url(../_css/design.css);
    </style>
</head>
<body>
    <div id="header">
        <a href=" ../Default.aspx"></a>
    </div>
    <form id="Form1" runat="server">
    <div id="navigation">
        <ul>
            <li class="first">
                <asp:loginstatus ID="Loginstatus1" runat="server"></asp:loginstatus>
            </li>
            <asp:LoginView ID="LoginView1" runat="server">
                <AnonymousTemplate>
                    <li><a href=" ../Register.aspx">Register</a></li>
                </AnonymousTemplate>
                <LoggedInTemplate>
                    <li><a href="Blog.aspx">My Blog</a></li>
                </LoggedInTemplate>
            </asp:LoginView>
        </ul>
    </div>
    <div id="content">
        <asp:SqlDataSource ID="SqlDataSource1" runat="server"
            ConnectionString="<%%$ ConnectionStrings:ConnectionString %>"
            SelectCommand="SELECT * FROM [Blogs] WHERE ([UserID] = @UserID) "
            DataSourceMode="DataReader">
            <SelectParameters>
                <asp:Parameter Name="UserID" Type="String" />
            </SelectParameters>
        </asp:SqlDataSource>
        <div class="section">
            <h1>Welcome to Your Blog: <asp:Label ID="BlogName" runat="server" Text=""></
asp:Label>
            !</h1>
            <p><a href="BlogPost.aspx">Make a new post.</a></p>
            <p><asp:Label ID="Permalink" runat="server" Text="Label"></asp:Label></p>
        </div>
        <div class="section">
            <h1>Your Recent Blog Posts</h1>
            <asp:Label ID="RecentPosts" runat="server" Text=""></asp:Label>
        </div>
    </div>
    </form>
    <div id="footer">
        Copyright &copy 2009, Brett Bryant, all rights reserved.
    </div>
</body>
</html>

```



http://localhost:60525/Blogger.naut/user/BlogPost.aspx

Blogger.naut: Blog: Post

(v) Lorem Ipsum - All the facts - Ipsum ge...



Blogger.naut

Logout

My Blog

Make a new post...

Title

SENIOR PROJET POR

Content

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus in nibh tortor. Suspendisse potenti. Praesent non tincidunt lectus. Donec id lorem ac nisi pharetra condimentum ut a magna. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec consequat hendrerit eros in placerat. Etiam ut quam nisl, tempor interdum elit. Curabitur imperdiet tristique odio sit amet ultricies.

Post

Done

```
' Brett Bryant
' -----
' Program name: Bloggernaut
'
' Created:      November 9th, 2009
' Last Modified: November 16th, 2009
'
' Course:      COIN 325 Java I
' Grade Received: A
' -----
'
' Comments:
'     This page is responsible for three things:
'     1) Rerouting people without blogs before the
'         page starts to load to the BlogCreate.aspx
'         form to create their blog before trying
'         to post.
'
'     2) Posting a blog entry after the submit button
'         is clicked from BlogPost.aspx.
'
'     3) Updating the LastUpdated column in the blogs
'         table as a post into that blog is being made.
'
' Security Concerns:
'     The implementation of both user/BlogPost and ViewBlog
'     invite XSS (HTML Injection) attacks because there's no
'     attempt to sanitize input from containing HTML.
'
Imports System
Imports System.Data
Imports System.Data.Sql
Imports System.Data.SqlClient

Partial Class user_BlogPost
    Inherits System.Web.UI.Page

    Dim BlogID As Integer
    Dim UserID As String

    ' When the page loads, use SqlDataSource1 to determine if
    ' the user visiting this page has created a blog or not.
    ' If they haven't, redirect them to the user/BlogCreate.aspx
    ' page.
    Protected Sub Page_Load(ByVal sender As Object, ByVal e As System.EventArgs) Handles Me.Load
        Dim reader As SqlDataReader

        UserID = Membership.GetUser().ProviderUserKey.ToString()
        SqlDataSource1.SelectParameters("UserID").DefaultValue = UserID
        reader = SqlDataSource1.Select(DataSourceSelectArguments.Empty)

        If Not reader.Read() Then
            Response.Redirect("BlogCreate.aspx")
        End If

        BlogID = reader("ID")
    End Sub

    ' "Post" button. Inserts the post information into the BlogPosts table
    ' and updates the LastUpdated column in the corresponding blog's row
    ' in the Blogs table.
    Protected Sub Button1_Click(ByVal sender As Object, ByVal e As System.EventArgs)
```

```
Handles Button1.Click
    SqlDataSource1.InsertParameters("BlogID").DefaultValue = BlogID
    SqlDataSource1.InsertParameters("UserID").DefaultValue = UserID
    SqlDataSource1.InsertParameters("UserEmail").DefaultValue = Membership.GetUser().✘
Email
    SqlDataSource1.InsertParameters("Title").DefaultValue = Title.Text
    SqlDataSource1.InsertParameters("Text").DefaultValue = Content.Text
    SqlDataSource1.UpdateParameters("UserID").DefaultValue = UserID
    SqlDataSource1.Insert()
    SqlDataSource1.Update()

    Response.Redirect("Blog.aspx")
End Sub
End Class
```

```

<%@ Page Language="VB" AutoEventWireup="false" CodeFile="BlogPost.aspx.vb" Inherits=
    "user_BlogPost" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head id="Head1" runat="server">
    <title>Bloggernaut: Blog: Post</title>
    <style type="text/css" media="screen">
        @import url(../_css/design.css);
    </style>
</head>
<body>
    <div id="header">
        <a href="../Default.aspx"></a>
    </div>
    <form id="Form1" runat="server">
<div id="navigation">
    <ul>
        <li class="first">
            <asp:loginstatus ID="Loginstatus1" runat="server"></asp:loginstatus>
        </li>
        <asp:LoginView ID="LoginView1" runat="server">
            <AnonymousTemplate>
                <li><a href="../Register.aspx">Register</a></li>
            </AnonymousTemplate>
            <LoggedInTemplate>
                <li><a href="Blog.aspx">My Blog</a></li>
            </LoggedInTemplate>
        </asp:LoginView>
    </ul>
</div>
<div id="content">
    <asp:SqlDataSource ID="SqlDataSource1" runat="server"
        ConnectionString="<%$ ConnectionStrings:ConnectionString %>"
        DataSourceMode="DataReader"
        SelectCommand="SELECT * FROM [Blogs] WHERE ([UserID] = @UserID)"
        InsertCommand="INSERT INTO BlogPosts(BlogID, UserID, UserEmail, DatePosted,
Title, Text) VALUES (@BlogID, @UserID, @UserEmail, GetDate(), @Title, @Text)"
        UpdateCommand="UPDATE [Blogs] SET [LastUpdated] = GetDate() WHERE ([UserID] =
@UserID)">
        <SelectParameters>
            <asp:Parameter Name="UserID" Type="String" />
        </SelectParameters>
        <InsertParameters>
            <asp:Parameter Name="BlogID" />
            <asp:Parameter Name="UserID" />
            <asp:Parameter Name="UserEmail" />
            <asp:Parameter Name="Title" />
            <asp:Parameter Name="Text" />
        </InsertParameters>
        <UpdateParameters>
            <asp:Parameter Name="UserID" Type="String" />
        </UpdateParameters>
    </asp:SqlDataSource>
    <div class="section">
        <h1>Make a new post...</h1>
        <p>
            <table cellspacing="0" cellpadding="4" border="0">
                <tr>
                    <td width="30"><strong>Title</strong></td>
                    <td>
                        <asp:TextBox ID="Title" runat="server"></asp:TextBox>
                        <asp:RequiredFieldValidator ID="RequiredFieldValidator1"
runat="server"
                            ControlToValidate="Title" ErrorMessage="You must give a

```

```
title."></asp:RequiredFieldValidator>
    </td>
</tr>
<tr>
    <td><strong>Content</strong></td>
    <td>
        <asp:TextBox ID="Content" runat="server" Height="147px"
TextMode="MultiLine"
        Width="480px"></asp:TextBox>
        <asp:RequiredFieldValidator ID="RequiredFieldValidator2"
runat="server"
        ControlToValidate="Content" ErrorMessage="You must enter
some content."></asp:RequiredFieldValidator>
    </td>
</tr>
<tr>
    <td>&nbsp;</td>
    <td>
        <asp:Button ID="Button1" runat="server" Text="Post" /></td>
</tr>
</table>
</p>
</div>
</div>
</form>
<div id="footer">
    Copyright &copy; 2009, Brett Bryant, all rights reserved.
</div>
</body>
</html>
```



http://localhost:60525/Blogger.naut/ViewBlog.aspx?ID=4

Blogger.naut: View Blog

[v] Lorem Ipsum - All the facts - Ipsum ge...

Blogger.naut

Logout

My Blog

portfolio

Recent Blog Posts

- 'SENIOR PROJET PORTFOLIO' posted on 4/9/2010 4:09:15 AM

Copyright © 2009, Blogger.naut, all rights reserved.

Done

```

' Brett Bryant
' -----
' Program name: Bloggernaut
' Created:      November 9th, 2009
' Last Modified: November 16th, 2009
'
' Course:      COIN 325 Java I
' Grade Received: A
' -----

```

```

' Comments:
'   This page is responsible for displaying
'   the posts of a blog. It makes sure that
'   client provided it with a valid blog ID,
'   otherwise it redirects to Default.aspx.
'   Otherwise, it writes every post made in
'   that blog into an unordered list for
'   users to choose from.
'

```

```

Imports System
Imports System.Data
Imports System.Data.Sql
Imports System.Data.SqlClient

```

```

Partial Class ViewBlog
    Inherits System.Web.UI.Page

```

```

Protected Sub Page_Load(ByVal sender As Object, ByVal e As System.EventArgs) Handles Me.Load
    Dim reader As SqlDataReader

    If Request.QueryString("ID") = Nothing Then
        Response.Redirect("Default.aspx")
    End If

    SqlDataSource1.SelectParameters("ID").DefaultValue = Integer.Parse(Request.QueryString("ID"))
    reader = SqlDataSource1.Select(DataSourceSelectArguments.Empty)

    If reader.Read() Then
        BlogName.Text = reader("BlogName")

        SqlDataSource2.SelectParameters("BlogID").DefaultValue = reader("ID")
        reader = SqlDataSource2.Select(DataSourceSelectArguments.Empty)

        If reader.Read() Then
            RecentPosts.Text = "<ul>"

            Do
                RecentPosts.Text += "<li><a href='ViewPost.aspx?ID=" + reader("ID").ToString() + "'>" + reader("Title") + "</a> posted on " + reader("DatePosted").ToString() + "</li>"
            Loop While reader.Read()

            RecentPosts.Text += "</ul>"
        Else
            RecentPosts.Text = "<p>It doesn't look like this blog has made any posts yet.</p>"
        End If
    Else
        Response.Redirect("Default.aspx?error=1")
    End If

```

End Sub
End Class

```
<%@ Page Language="VB" AutoEventWireup="false" CodeFile="ViewBlog.aspx.vb" Inherits="
    "ViewBlog" Debug="true" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head id="Head1" runat="server">
    <title>Bloggernaut: View Blog</title>
    <style type="text/css" media="screen">
        @import url(_css/design.css);
    </style>
</head>
<body>
    <div id="header">
        <a href="Default.aspx"></a>
    </div>
    <div id="navigation">
        <ul>
            <li class="first">
                <form id="Form1" runat="server">
                    <asp:loginstatus ID="Loginstatus1" runat="server"></asp:loginstatus>
                </form>
            </li>
            <asp:LoginView ID="LoginView1" runat="server">
                <AnonymousTemplate>
                    <li><a href="Register.aspx">Register</a></li>
                </AnonymousTemplate>
                <LoggedInTemplate>
                    <li><a href="user/Blog.aspx">My Blog</a></li>
                </LoggedInTemplate>
            </asp:LoginView>
        </ul>
    </div>
    <div id="content">
        <asp:SqlDataSource ID="SqlDataSource1" runat="server"
            ConnectionString="<%= ConnectionStrings:ConnectionString %>"
            SelectCommand="SELECT * FROM [Blogs] WHERE ([ID] = @ID)"
            DataSourceMode="DataReader">
            <SelectParameters>
                <asp:Parameter Name="ID" Type="Int32" />
            </SelectParameters>
        </asp:SqlDataSource>
        <asp:SqlDataSource ID="SqlDataSource2" runat="server"
            ConnectionString="<%= ConnectionStrings:ConnectionString %>"
            SelectCommand="SELECT * FROM [BlogPosts] WHERE ([BlogID] = @BlogID)"
            DataSourceMode="DataReader">
            <SelectParameters>
                <asp:Parameter Name="BlogID" Type="Int32" />
            </SelectParameters>
        </asp:SqlDataSource>
        <div class="section">
            <h1><asp:Label ID="BlogName" runat="server" Text=""></asp:Label></h1>
            <h3>Recent Blog Posts</h3>
            <asp:Label ID="RecentPosts" runat="server" Text=""></asp:Label>
        </div>
    </div>
    <div id="footer">
        Copyright &copy 2009, Brett Bryant, all rights reserved.
    </div>
</body>
</html>
```



http://localhost:60525/Blogger.naut/ViewPost.aspx?ID=7

Blogger.naut: View Blog Post

[x] Lorem Ipsum - All the facts - Ipsum ge...

Blogger.naut

Logout

My Blog

portfolio : SENIOR PROJET PORTFOLIO

Posted on: 4/9/2010 4:08:15 AM

» Go back to the 'portfolio' blog.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus in nibh tortor. Suspendisse potenti. Praesent non tincidunt lectus. Donec vestibulum tempus portitor sem vel luctus. Fusce accumsan pellentesque urna, ac semper est suscipit eget. Proin sagittis, eros non i

```

<%@ Page Language="VB" AutoEventWireup="false" CodeFile="ViewPost.aspx.vb" Inherits=
    "ViewPost" Debug="true" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head id="Head1" runat="server">
    <title>Bloggernaut: View Blog Post</title>
    <style type="text/css" media="screen">
        @import url(_css/design.css);
    </style>
</head>
<body>
    <div id="header">
        <a href="Default.aspx"></a>
    </div>
    <div id="navigation">
        <ul>
            <li class="first">
                <form id="Form1" runat="server">
                    <asp:loginstatus ID="Loginstatus1" runat="server"></asp:loginstatus>
                </form>
            </li>
            <asp:LoginView ID="LoginView1" runat="server">
                <AnonymousTemplate>
                    <li><a href="Register.aspx">Register</a></li>
                </AnonymousTemplate>
                <LoggedInTemplate>
                    <li><a href="user/Blog.aspx">My Blog</a></li>
                </LoggedInTemplate>
            </asp:LoginView>
        </ul>
    </div>
    <div id="content">
        <asp:SqlDataSource ID="SqlDataSource1" runat="server"
            ConnectionString="<%%$ ConnectionStrings:ConnectionString %>"
            SelectCommand="SELECT * FROM [BlogPosts] WHERE ([ID] = @ID)"
            DataSourceMode="DataReader">
            <SelectParameters>
                <asp:Parameter Name="ID" Type="Int32" />
            </SelectParameters>
        </asp:SqlDataSource>
        <asp:SqlDataSource ID="SqlDataSource2" runat="server"
            ConnectionString="<%%$ ConnectionStrings:ConnectionString %>"
            SelectCommand="SELECT * FROM [Blogs] WHERE ([ID] = @BlogID)"
            DataSourceMode="DataReader">
            <SelectParameters>
                <asp:Parameter Name="BlogID" Type="Int32" />
            </SelectParameters>
        </asp:SqlDataSource>
        <div class="section">
            <h1><asp:Label ID="BlogName" runat="server" Text=""></asp:Label>;
            <asp:Label ID="PostTitle" runat="server" Text=""></asp:Label></h1>
            <p>Posted on: <asp:Label ID="DatePosted" runat="server" Text=""></asp:Label>
        </p>
            <p><asp:Label ID="BlogLink" runat="server" Text=""></asp:Label></p>
            <p><asp:Label ID="PostText" runat="server" Text=""></asp:Label></p>
        </div>
    </div>
    <div id="footer">
        Copyright &copy 2009, Brett Bryant, all rights reserved.
    </div>
</body>
</html>

```

```
' Brett Bryant
```

```
' -----  
' Program name: Bloggernaut
```

```
' Created:          November 9th, 2009
```

```
' Last Modified:    November 16th, 2009
```

```
' Course:           COIN 325 Java I
```

```
' Grade Received:   A  
' -----
```

```
' Comments:
```

```
'     This page is responsible for viewing a post  
'     on a blog. It redirects to Default.aspx when  
'     it was requested from a client who didn't  
'     provide a post ID or when that post ID does  
'     not exist. Otherwise post data shows up as  
'     expected.  
'
```

```
Imports System
```

```
Imports System.Data
```

```
Imports System.Data.Sql
```

```
Imports System.Data.SqlClient
```

```
Partial Class ViewPost
```

```
    Inherits System.Web.UI.Page
```

```
    Protected Sub Page_Load(ByVal sender As Object, ByVal e As System.EventArgs) Handles Me.Load
```

```
        Dim reader As SqlDataReader
```

```
        If Request.QueryString("ID") = Nothing Then
```

```
            Response.Redirect("Default.aspx")
```

```
        End If
```

```
        SqlDataSource1.SelectParameters("ID").DefaultValue = Request.QueryString("ID")  
        reader = SqlDataSource1.Select(DataSourceSelectArguments.Empty)
```

```
        If reader.Read() Then
```

```
            DatePosted.Text = reader("DatePosted")
```

```
            PostTitle.Text = reader("Title")
```

```
            PostText.Text = "<pre>" + reader("Text").ToString() + "</pre>"
```

```
            SqlDataSource2.SelectParameters("BlogID").DefaultValue = reader("BlogID")
```

```
            reader = SqlDataSource2.Select(DataSourceSelectArguments.Empty)
```

```
            reader.Read()
```

```
            BlogName.Text = reader("BlogName")
```

```
            BlogLink.Text = "> <a href='ViewBlog.aspx?ID=" + reader("ID").ToString() +  
">Go back to the '" + reader("BlogName") + "' blog.</a>"
```

```
        Else
```

```
            Response.Redirect("Default.aspx?error=2")
```

```
        End If
```

```
    End Sub
```

```
End Class
```

[illegible]

Max number in queue: 15 customers

Max time in queue: 36 minutes

brett@brett-laptop:~\$

```

/*
 * Brett Bryant
 *
 * -----
 * Program name:    Supermarket Queuing Simulation,
 *                  Assignment 15
 * Created:         Fall Semester, 2007
 * Last Modified:   Fall Semester, 2007
 *
 * Course:         COIN 325 Java I
 * Grade Received:  A
 *
 * -----
 *
 * Comments:
 *     This java application attempts to simulate
 *     a busy supermarket queue where between every
 *     0-3 iterations (mins) a customer could arrive
 *     or leave from the supermarke queue.
 *
 * Example:
 *     # java Supermarket
 *     A customer has ENTERED the queue...
 *     A customer has LEFT the queue...
 *     A customer has LEFT the queue...
 *     A customer has ENTERED the queue...
 *     A customer has ENTERED the queue...
 *     (...)
 *     A customer has LEFT the queue...
 *     A customer has ENTERED the queue...
 *     A customer has LEFT the queue...
 *     A customer has ENTERED the queue...
 *     A customer has LEFT the queue...
 *     Max number in queue: 12 customers
 *     Max time in queue: 30 minutes
 */
import java.util.Random;

class Supermarket {
    private static final Random rand = new Random();

    private static final int LEAVE_INTERVAL = 4;
    private static final int ARRIVAL_INTERVAL = 4;

    public static void main(String args[]) {
        int maxQueue = 0, // Maximum number of people to have ever been in queue
        in queue      maxTimeInQueue = 0, // Maximum amount of time any one individual has spent
        at a time     arrivalTime = 0, // The time until the next arrival into the queue
        leaveTime = -1, // The time until somebody leaves the Queue
        timeInQueue = 0; // Temporary variable for one customer's time in queue

        CustomerQueue line = new CustomerQueue();

        // Choose a random arrival time for the first member
        // of the queue between [0, ARRIVAL_INTERVAL)
        arrivalTime = rand.nextInt(ARRIVAL_INTERVAL);

        // Simulate the queue for 720 minutes (12 hours)
        for (int i=0; i<720; i++) {

            // Check if somebody is arriving at the queue this iteration,
            // while simultaneously decrementing arrivalTime for the
            // next iteration to count down time for next arrival in
            // case there were no arrivals this iteration.
            if (arrivalTime-- == 0) {
                // The first time somebody arrives in the queue, calculate
                // the time until the customer leaves between [0, LEAVE_INTERVAL)
                if (leaveTime < 0) leaveTime = rand.nextInt(LEAVE_INTERVAL);
                System.out.printf("A customer has ENTERED the queue...\n");
                arrivalTime = rand.nextInt(ARRIVAL_INTERVAL);
            }
        }
    }
}

```

```
/*
 * Brett Bryant
 * -----
 * Program name:      Supermarket Queuing Simulation
 *                   (Customer class), Assignment 15
 * Created:           Fall Semester, 2007
 * Last Modified:     Fall Semester, 2007
 *
 * Course:            COIN 325 Java I
 * Grade Received:    A
 * -----
 *
 * Comments:
 *   This Customer class is very small. It's only
 *   function is to keep track of how many iterations
 *   occurred in the simulation when the customer
 *   entered the supermarket queue. This is used later
 *   as the Customer is removed from the queue and
 *   getTime() is subtracted from the current number
 *   of iterations to find out how many iterations
 *   they spent waiting.
 */
public class Customer {
    private int time;

    public Customer (int time) {
        this.time = time;
    }

    public int getTime() {
        return this.time;
    }
}
```

```
        line.push(new Customer(i));
    }

    // Check if somebody is leaving from the queue this iteration,
    // while simultaneously decrementing leaveTime for the next
    // iteration to count down time until the next departure in
    // case there were no departures this iteration.
    if (leaveTime-- == 0) {
        System.out.printf("A customer has LEFT the queue...\n");

        // The try..catch block is used here in case the frequency
        // of departures becomes out of pace with arrivals and
        // the queue is empty.
        try {
            leaveTime = rand.nextInt(LEAVE_INTERVAL);
            timeInQueue = i - line.pop().getTime();
        } catch (Exception e) { continue; }

        // This line is safe after the try...catch because
        // timeInQueue can only be two things: It's initial
        // value 0, or the time of a customer that has left
        // the queue. In the event that departures are being
        // too frequent for arrivals, the exception will be
        // thrown when the line is empty and no assignment
        // to timeInQueue will take place that iteration.
        maxTimeInQueue = Math.max(maxTimeInQueue, timeInQueue);
    }

    // Keep track of the largest the line has been
    maxQueue = Math.max(maxQueue, line.size());
}

System.out.printf("Max number in queue: %d customers\n", maxQueue);
System.out.printf("Max time in queue: %d minutes\n", maxTimeInQueue);
}
```

```
/*
 * Brett Bryant
 * -----
 * Program name:      Supermarket Queuing Simulation
 *                   (CustomerQueue class), Assignment 15
 * Created:           Fall Semester, 2007
 * Last Modified:     Fall Semester, 2007
 *
 * Course:            COIN 325 Java I
 * Grade Received:    A
 * -----
 *
 * Comments:
 *   The CustomerQueue class creates a Queue of
 *   customers using a java ArrayList of Customer
 *   objects. Basic queue functions are implemented:
 *   push, pop, isEmpty and size.
 */

import java.util.List;
import java.util.ArrayList;

public class CustomerQueue {
    private List<Customer> cq = new ArrayList<Customer>();

    public void push(Customer customer) {
        cq.add(customer);
    }

    public Customer pop() throws IndexOutOfBoundsException {
        return cq.remove(0);
    }

    public boolean isEmpty() {
        return cq.isEmpty();
    }

    public int size() {
        return cq.size();
    }
}
```

 **brett@brett-laptop: ~**

File Edit View Terminal Help

brett@brett-laptop:~\$./problem-1.pl 1219519

1219519: one million two hundred nineteen thousand five hundred nineteen

brett@brett-laptop:~\$./problem-2.pl

Two files were attempted, one was successful.

brett@brett-laptop:~\$ 

```
#!/usr/bin/perl -w

#
# Brett Bryant
# -----
# Program name: Problem 1, Homework 10
# Created:      Fall Semester, 2008
# Last Modified: Fall Semester, 2008
#
# Course:       COIN 333 Applied Systems
# Grade Received: A
# -----
#
# Comments:
#   This perl script checks for at least one
#   command line argument, and if it's called
#   with one it will then print that argument
#   along with the conversion of the
#   argument's integer representation into
#   english words.
#
# Example:
#   # problem-1.pl 103
#   103: one hundred three
#
use numToWords;

print "$ARGV[0]: " . numToWords::convert($ARGV[0]) . "\n" if $#ARGV >= 0;
```

```
#!/usr/bin/perl -w

#
# Brett Bryant
# -----
# Program name:      Problem 2, Homework 10
# Created:           Fall Semester, 2008
# Last Modified:     Fall Semester, 2008
#
# Course:            COIN 333 Applied Systems
# Grade Received:    A
# -----
#
# Comments:
#   This program determines the ratio of *.cpp files
#   in the current directory that will compile
#   successfully using g++, to those that wont.
#
#   Afterwards, it prints out a resulting message without
#   using numerals in the following format..
#
#       - X file(s) was/were attempted, Y files was/were successful.
#
#       - e.g. Four files were attempted, one was successful.
#       - e.g. One file was attempted, zero were successful.
#
#
use numToWords;

my $attempt = 0;
my $success = 0;

for $sourceFile (<*.cpp>) {
    $file = $sourceFile;
    $file =~ s/\.cpp$//i;

    unlink "$file.errors"
        if not system("g++ -o $file $sourceFile 2> $file.errors");

    $attempt++;
    $success++ if not $?;
}

$attemptStr = numToWords::convert($attempt);
$attemptStr =~ s/(.)/uc($1)/e;
print "$attemptStr file";
print "s" if $attempt != 1;
print " w" . ("as", "ere")[$attempt != 1];
print " attempted, " . numToWords::convert($success) . " w" . ("as", "ere")[$success != 1];
print " successful.\n";
```

```
#!/usr/bin/perl -w

#
# Brett Bryant
# -----
# Program name:      numToWords package, Homework 10
# Created:           Fall Semester, 2008
# Last Modified:      Fall Semester, 2008
#
# Course:            COIN 333 Applied Systems
# Grade Received:     A
# -----
#
# Comments:
#   This perl package provides a single
#   subroutine which converts an integer into
#   string of corresponding english words.
#
# Example:
#   - e.g. numToWords::convert(101) eq "one hundred one"
#   - e.g. numToWords::convert(3197) eq "three thousand one hundred ninety seven"
#
package numToWords; {
    sub convert {
        my $num = shift;
        my $mod = 10;
        my $negative = 0;
        my @digits = (
            "",
            "one",
            "two",
            "three",
            "four",
            "five",
            "six",
            "seven",
            "eight",
            "nine",
        );
    };

    my %postfix = (
        1000      => "thousand",
        1000000    => "million",
        1000000000 => "billion",
        1000000000000 => "trillion",
    );

    my %tens = (
        10 => "ten",
        11 => "eleven",
        12 => "twelve",
        13 => "thirteen",
        14 => "fourteen",
        15 => "fifteen",
        16 => "sixteen",
        17 => "seventeen",
        18 => "eighteen",
        19 => "nineteen",
        20 => "twenty",
        30 => "thirty",
        40 => "fourty",
        50 => "fifty",
        60 => "sixty",
        70 => "seventy",
        80 => "eighty",
        90 => "ninety",
    );
};
```

```

my @lex = ();

if ($num < 0) {
    $negative = 1;
    $num = abs $num;
} elsif ($num == 0) {
    return 'zero';
}

while ($num > 0) {
    my $n = $num % $mod;

    if ($mod == 10) {
        if ($n && (($num % 100) < 11 || ($num % 100) > 19)) {
            unshift @lex, $digits[$n];
        } else {
            $num += $n;
        }
    } elsif ($mod == 100) {
        unshift @lex, $tens[$num % $mod] if $num % $mod;
        if (($num / $mod) % 10) {
            unshift @lex, ($digits[(($num / $mod) % 10), 'hundred'];
            $num -= 100 * ((($num / $mod) % 10);
        }
    } elsif ((($num / $mod) % 1000) {
        if (exists $postfix{$mod}) {
            unshift @lex, $postfix{$mod};
            unshift @lex, numToWords::convert((($num / $mod) % 1000);
        } else {
            unshift @lex, $num;
        }
    }

    $mod *= 100 if $mod > 100;
    $mod *= 10;
    $num -= $n;
}

unshift @lex, 'negative' if $negative;
return join ' ', @lex;
}

return 1;
}

```

❏ brett@brett-laptop: ~

File Edit View Terminal Help

brett@brett-laptop:~\$./testIndexedMultilist

1

0

0

insert(1): 1

55 A

55: 55

1

256

1

1

254 C

255 B

256 A

remove b: 1

254 C

256 A

1

254 C

254: 254

C 254: 20

20 t

19 s

18 r

17 q

16 p

15 o

14 n

13 m

```

/*
 * Brett Bryant
 * -----
 * Program name:    Indexed Multi List / Project 2
 * Created:         Spring Semester, 2008
 * Last Modified:   Spring Semester, 2008
 *
 * Course:         COIN 315 Data Structures
 * Grade Received:  B+ in COIN 315
 * -----
 *
 * Comments:
 *     This is the collection of definitions for the
 *     members of the IndexMultiList class. The
 *     IndexMultiList class implements a singly linked
 *     list between individual characters that can be
 *     modified, removed, and searched like a normal
 *     linked list implementatoin except functions
 *     for utilizing these things with consideration
 *     to the index of list nodes are provided as well.
 */

#include <iostream>
#include "IndexedMultiList.h"

using namespace std;

// Macro for checking wether Index is in a valid range
// returns true if: 0 <= i < DEFAULT_MAX_SIZE
// returns false otherwise
// #define validRange(i) (0 <= i && i < DEFAULT_MAX_SIZE)
// #define IML_ITER_LIST(name) for(Node *(name) = head; (name); (name) = (name)->next)

// IndexedMultiList constructor
IndexedMultiList::IndexedMultiList() {
    head = NULL; // set head to NULL
}

// IndexedMultiList deconstructor
// deletes all iteams from the list
IndexedMultiList::~IndexedMultiList() {
    for (Node *tmp, *cur = head; cur;) {
        tmp = cur->next;
        delete cur;
        cur = tmp;
    }
}

// returns true if there is no head element
bool IndexedMultiList::empty() {
    return !head;
}

// returns the sum of all count values
// stored in the list
int IndexedMultiList::size() {
    int totalSize = 0;

    for (Node *cur = head; cur; cur = cur->next)
        totalSize += cur->count;

    return totalSize;
}

// unique size is the number of elements in the list
int IndexedMultiList::uniqueSize() {
    int size = 0;
    for (Node *cur = head; cur; cur = cur->next, size++);
}

```

```

    return size;
}

// finds the first element in the list
// who's ->val = the first argument
int IndexedMultiList::find(char val) {
    int i = 0;

    for(Node *cur = head; cur; cur = cur->next) {
        if(cur->val == val)
            return i;
        i++;
    }

    return -1;
}

// returns the count of items in the first
// element of list where ->val = the first
// argument
int IndexedMultiList::countItem(char val) {
    return countPosition(find(val));
}

// returns the number of items at index "index"
// or 0 if index is not occupied or is out of range
int IndexedMultiList::countPosition(int index) {
    Node *cur;
    return (cur = getNodeAt(index)) ? cur->count : 0;
}

// returns the node at an index
// or NULL if it doesn't exist
IndexedMultiList::Node *IndexedMultiList::getNodeAt(int index) {
    Node *cur = head;

    if (index < 0)
        return NULL;

    while(index-- && cur)
        cur = cur->next;

    return cur;
}

// returns false if index is not in a valid range
// or the index position is not in use
// otherwise: sets argument 2 to the value inside
// of the element list_[index]
bool IndexedMultiList::getAt(int index, char &out) {
    Node *cur;
    if (!(cur = getNodeAt(index)))
        return false;

    out = cur->val;
    return !!cur->count;
}

// prints IML list data
void IndexedMultiList::print() {
    for (Node *cur = head; cur; cur = cur->next)
        cout << cur->count << " " << cur->val << endl;
}

// inserts IML data "val", "count" at "index"
bool IndexedMultiList::insert(int index, char val, int count) {
    Node *cur;

    if (index < 0)

```

```

        return false;

    if (!index)
        head = new Node(count, val, head);
    else if (index >= uniqueSize())
        getNodeAt(uniqueSize()-1)->next = new Node(count, val);
    else {
        cur = getNodeAt(index);
        cur = new Node(count, val, cur);
    }

    return true;
}

// adds "amount" to list_[index] if index is valid and
// in use... if addition results in the amount becoming
// <= 0 .. element is removed
bool IndexedMultiList::change(int index, int amount) {
    Node *cur;

    if (!(cur = getNodeAt(index)))
        return false;

    cur->count += amount;

    if (cur->count <= 0)
        remove(index);

    return true;
}

// Same as "change" except finds the element by searching
// by it's val, and updates the first element found
bool IndexedMultiList::update(char val, int amount) {
    return change(find(val), amount);
}

// removes the first element in list with the same
// val as the first argument
bool IndexedMultiList::remove(char val) {
    return (remove(find(val)) != -1);
}

// removes the index
int IndexedMultiList::remove(int index) {
    Node *cur, *tmp;
    int returnCount;

    if (!(cur = getNodeAt(index)))
        return -1;

    returnCount = cur->count;

    if (!index)
        head = head->next;
    else {
        (tmp = getNodeAt(index-1))->next = cur->next;
    }

    delete cur;

    return returnCount;
}

// removes all elements who's val is = first
// argument
int IndexedMultiList::removeAll(char val) {
    int i, c = 0;

```

```
    while ((i = find(val)) != -1)
        c += remove(i);
    return c;
}
```

```

/*
 * Brett Bryant
 * -----
 * Program name:    Indexed Multi List / Project 2
 * Created:         Spring Semester, 2008
 * Last Modified:   Spring Semester, 2008
 *
 * Course:         COIN 315 Data Structures
 * Grade Received:  B+ in COIN 315
 * -----
 *
 * Comments:
 *     This is the collection of prototypes for the
 *     IndexMultiList class.
 *
 *     The IndexMultiList class implements a singly linked
 *     list between individual characters that can be
 *     modified, removed, and searched like a normal
 *     linked list implementatoin except functions
 *     for utilizing these things with consideration
 *     to the index of list nodes are provided as well.
 */

const int DEFAULT_MAX_SIZE=20;

class IndexedMultiList {
public:
    //Constructors/destructors
    IndexedMultiList(); //creates an empty list capable of storing
    //DEFAULT_MAX_ITEMS items
    ~IndexedMultiList(); //destructor ONLY if needed

    //accessors
    bool empty(); //returns true iff (if and only if) list is empty
    int uniqueSize(); //returns the number of positions in the list in use
    int size(); //returns the total number of items in the list (using counts)
    //If 1 A, 3 C, 1 D are inserted into an IndexedMultiList iml then
    //impl.uniqueSize() would return 3 and impl.size() would return 5.
    int find(char val); //returns index of position with val in it or -1 if not found
    int countItem(char val); //returns number of times val is in the list (vacuously 0 if not found)
    int countPosition(int index); //returns number of items at position index (0 based), 0 if bad index
    bool getAt(int index, char &out);
    //if position index is in use, out is set to char at that position and true is returned, otherwise false is returned.
    void print(); //prints out the content of the list in order by position, see below

    //mutators
    bool insert(int index, char val, int count);
    /*insert count number of val into the list at position index
    returns true iff val was able to fit in list and index was valid and count >=1
    valid index is 0 up to size (inclusive). So for an empty list, 0 is the only valid index.
    insert should do nothing if index is invalid
    If the list has 3 A's in position 1, then 0 and 1 are the only valid indices.
    */
    bool change(int index, int amount);
    /*add amount to item in position index
    returns true iff index was valid
    insert should do nothing if index is invalid
    amount could be negative, if adding amount to the current count results in a count of
    <= 0 then
    position should be removed
    */

```

```
    bool update(char val, int amount); //finds val in list and adds amount (which could be negative)
    //return true iff found

    bool remove(char val); //removes 1 instance of val from list, returns true iff val was in list

    int remove(int index); //remove position at index, returns # removed or -1 if index not in use

    int removeAll(char val); //removes all instances of val from list
    //returns number of items removed, which could be 0
private:
    struct Node {
        Node(int c, char v, Node *n = NULL) :
            count(c), val(v), next(n) {};
        int count;
        char val;
        struct Node *next;
    } *head;

    // returns the node at an index in the linked list
    // or NULL if it does not exist
    Node *getNodeAt(int index);
    //put whatever methods or data you need in here,
    //you must put the 1-dimensional array here
};
```

```

/*
 * Brett Bryant
 * -----
 * Program name:    test for Indexed Multi List class / Project 2
 * Created:         Spring Semester, 2008
 * Last Modified:   Spring Semester, 2008
 *
 * Course:         COIN 315 Data Structures
 * Grade Received:  B+ in COIN 315
 * -----
 *
 * Comments:
 *     This is the collection of tests which aim to
 *     check the accuracy of the IndexMultiList class.
 *     Provided in the comments are the expected
 *     results of each time we print something to
 *     standard output as a sanity check.
 */

#include <iostream>
#include "IndexedMultiList.h"

using namespace std;

int main(int argc, char *argv[]) {
    IndexedMultiList iml;

    // Check sanity, should print 1,0,0
    cout << iml.empty() << endl;
    cout << iml.size() << endl;
    cout << iml.uniqueSize() << endl;

    // should print 1
    cout << "insert(1): " << iml.insert(0, 'A', 55) << endl;
    iml.print();
    // should print 55
    cout << "55: " << iml.countPosition(iml.find('A')) << endl;

    // 1
    cout << iml.change(iml.find('A'), 201) << endl;
    // 256
    cout << iml.countPosition(iml.find('A')) << endl;
    // 1
    cout << iml.insert(0, 'B', 0xff) << endl;
    // 1
    cout << iml.insert(0, 'C', 0xfe) << endl;
    // C 254
    // B 255
    // A 256
    iml.print();
    // 1
    cout << "remove b: " << iml.remove('B') << endl;
    // C 254
    // A 256
    iml.print();
    // 1
    //
    cout << iml.remove('A') << endl << endl;
    // C 254
    iml.print();
    // 254
    cout << "254: " << iml.remove(iml.find('C')) << endl;
    // C 254
    cout << "C 254: ";
    iml.print();

    for (char c = 'a'; c < 'a' + 20; c++)
        iml.insert(0, c, c - 'a' + 1);
}

```

```
cout << iml.uniqueSize() << endl;  
iml.print();
```

```
// remove 'b', 'e', 'a', 't'  
iml.remove(iml.find('b'));  
iml.remove(iml.find('e'));  
iml.remove(iml.find('a'));  
iml.remove(iml.find('t'));
```

```
cout << endl;  
iml.print();
```

```
// 16  
cout << iml.uniqueSize() << endl;
```

```
cout << endl;
```

```
return 0;
```

```
}
```

Papers



Ethical Internet Gambling (COIN 315 Data Structures)



This paper looks at the ethical implications of gambling while on the internet. It examines this through discussing the legality, consequences, and christian morals involved.



A Theoretical and Practical Discussion of The Privacy of Personal Data in Web Applications (COIN 333 Applied Networking)



This topic is discussed by looking at what legal protections you have online, and where the pitfalls are in internet security that can confuse even the least modest of people.



Data Thieves and Information Security (COIN 330 Architecture)



Data Thieves and Information Security looks briefly at the topic of botnets, and then goes on to ponder about the responsibility of information security. It also mentions a few other ways you can protect yourself from data thieves, and what suprisingly common problems usually leave a network vulnerable.

Bryant 1

Brett Bryant

Professor Underwood

Coins: Data Structures

January 28th, 2008

Ethical Internet Gambling:

Can You Ethically Emulate a Real Casino Online?

Gambling has been a staple social activity of human existence for as long as people have had things to trade. In places in the United States casino gambling is legal and there are conflicting disagreements about whether or not these places should allow games of chance to be played on the internet as well. Even though online gambling conceptually can't be any less ethical than gambling, many factors make the realm of the gambling online considered unsafe and a possible haven for internet criminals. While it's very hard to cheat at an online poker game, there's nobody checking to see if you're being cheated.

Right now, the only standardizing law for internet gambling in the United States prohibits betting on sporting events and games of non-chance (Wikipedia). The decision by the Supreme Court that prohibits games of non-chance to be played applied retroactively to online gambling (Wikipedia). However, only a few states explicitly deny the ability to legally play games like poker online in their states. These states are: Illinois, Indiana, Louisiana, Montana, Nevada, Oregon, South Dakota, and Washington (Humphrey).

Even though a common misconception on the ban of online gambling from places like Nevada is because so much of their economy revolves around gambling casinos that the prosperity of the state's economy might be ruined unless it removes online gambling. Although these arguments might have some truth; the sinister reality of online gambling sites are that there is no regulatory bodies to govern them, as they have in Nevada, nor any proof from the card dealer that the computer isn't stacking the deck or taking advantage of you.

There are a number of different techniques that an administrator of an online gaming site could use to trick you into believing there's a fair game being played. A couple possible ideas could be placing their own players inside of games who would have better chances at the cards than you. When done on such a small scale such an attempt to defraud consumers would only be detectable around statistical improbability when somebody has access to the complete statistics of the site, which would never even be kept by a dishonest company. Besides slipping a thief into the game, they could write algorithms to follow a statistically random average but be more likely to take your money on a larger bet. Trusting your identity, money, or chances to a company that could have avoided registration by setting up in Taiwan is never a good idea either.

Stacking the deck or changing algorithms isn't the only concern about online gambling casinos either. Because of all the money changing hands on their sites daily, it makes it hard to track where stolen money has gone that has been spent on gambling. The concern isn't about retrieving the money, but preventing people from controlling where the money goes if it can't be tracked. Using stolen credit cards to fund a game that you throw away to your legitimate player that cash's the credits out may leave the theft completely untraceable.

Simplest to say; the best interest of the consumer is to avoid online gambling, or gambling altogether, but not to make gambling illegal altogether. The Bible talks in many places about a plan for us by god, and the will to change it or to try and change fate as being defiantly unchristian. "The Parable of the Talents (Matthew 25:14-30) indicates that the good and faithful servants administered the talents entrusted to them in such a way that the master was pleased. The wicked and slothful servant failed in his administration and suffered the appropriate consequences. (Gambling)"

Even though online gambling has it's inherent risk, games that aren't chance show a legitimate role teaching strategy and statistics. Although it's usually the government's responsibility to protect someone from themselves, the ability of people to make their own choices is a very valuable commodity. All people who know the allure of gambling inherently know the risks as well. The reasons casinos make money is because the odds are in their favor, and the fact they have patrons who are not as good at controlling their finances and habits as others is not the fault of casinos. The argument for the government to make casinos and online gambling illegal is similar to an argument where the government steps in to protect you from making a bad choice at a car lot if a dealer tells you that it's not a good deal. The reasoning for that comparison is that when the information about the risk of making any purchase is freely available to you, and you choose to ignore it then you may be less responsible than others to have the freedom to spend your own money, but when the money is yours you can spend the money however you want.

Works Cited

"Online Gambling - Wikipedia, the Free Encyclopedia." Wikipedia. 28 Jan. 2008. 28 Jan. 2008

<http://en.wikipedia.org/wiki/Online_gambling#Legality>.

Humphrey, Chuck. "State Gambling Law Summary." Gambling-Law-Us.Com. 28 Jan. 2008

<<http://www.gambling-law-us.com/State-Law-Summary/>>.

"Gambling: a Biblical Perspective." www.ag.org. 28 Jan. 2008

<http://www.ag.org/top/Beliefs/Position_Papers/pp_4186_gambling.cfm>.

**A Theoretical and Practical Discussion of
The Privacy of Personal Data in Web Applications**

Brett Bryant

Gary Underwood

COINS 333

September 10th, 2008

A Theoretical and Practical Discussion of
The Privacy of Personal Data in Web Applications

Since the change to digitization of your everyday life to digital data, many people have been considering that this gives power to people who may not be trusted. The core question comes from the proverb '*Quis custodiet custodes ipsos*' which means 'Who will watch the watchers?' Once you give your input to the computer, you're subjecting yourself to a chain of entities that have control over the information you send across the internet. When you sign up for a website, giving your personal information into a computer to register a product, or create a new email address the basic information that you give these websites is private information shared between the website and you. By believing that your data won't be used maliciously, we trust the people we give the information to enough to not do anything with it which we would disagree with, or legally follow their privacy policy on their website. As long as we can trust that data we pass out on networks does us more good than harm than we don't mind thinking about the proverbial watcher as much as we might if we had no security at all.

One of the biggest problems with online security is having users that can see something deceiving and trust it enough to enter their information into it. There are many places online where you can go to find clear examples of dishonorable unchristian ethical practices, and signs of this deception. For instance, a common deception of trust is to find a used car for sale at a ridiculously low price by an owner with an equally extravagant story like shipping of to war that want to use a third party for the protection of both parties in an attempt to get you to at least pay

them to ship the car to where you are, and hopefully steal more of your money if you agree to the third party option. The deceptions listed here have become an issue because people with low moral standards seem to believe those with a different level of understanding of what they should trust on the internet have less of a right to not being exploited of that trust. Leviticus chapter 19 verse 11 says “Ye shall not steal, neither deal falsely, neither lie one to another,” which instructs that people who read it try to understand the negativity in this kind of behavior.

One of the hardest things to recover from being a victim of is an issue with your credit report. Problems with your credit report will occur when other online scams appear online and you're lead to a place you think you trust to give this information too, or when a malicious user gains access to a place that you should have trusted to keep your data safe. To help consumers in the United States with determining if they have been a victim of fraud, the Fair and Accurate Credit Transactions Act of 2003, or FACTA, allows you to request a complimentary credit report once a year from one of the three major credit reporting agencies in the US (Fair and Accurate). Even when you've figured out that you've been a victim, it's still a thorough process which takes years to get your credit score back to where it should have been.

The responsibility of keeping your data secure once it's in the hands of a private company is another thing that has to be focused on very intensely because there is no way for the user to know whether or not the website that has their credit card number, and other sensitive information is doing something insecure with your data once they have it. The most common relational databases, which are what is used to store 99% of dynamically changing information given in websites, are equipped with the security features that when deployed properly drastically

decrease the chances of somebody gaining malicious access to that information. By default, a man in the middle attack on a remote MySQL database is as simple as watching the connection to the remote database server when no extra effort is done to setup SSL or some encryption layer (5.5.7). This one simple fact that a networking bridge along the path of the database server and the web server could lead to untraceable data theft, would make you believe that it alone has caused damage to credit reports of people who trusted them not to steal their personal information.

The rights that users on the internet have to protecting their data, when not speaking in terms of the health or education sectors which have their own laws regarding private information, are largely protected by who the user decides to give his information to and whether or not they themselves can be trustworthy. Even though it is illegal to possess private financial credentials of a person you haven't been authorized to use them by, these laws are hard to enforce because they require the person who is possessing them to be searched with reasonable suspicion, and that is usually some type of computer crime that they've already committed. There is a large ecosystem online for data like stolen credit card lists and unreleased security exploits where the people who bring you the modern definition of computer hacker, or cyber criminal, exchange them amongst their selves to eventually make a profit. Good technical practices have to be upheld and researched by those attempting to conceal their data from poisonous users who are committing cyber crimes against unsuspecting users.

When the private party who collects any kind of sensitive materials from a user asks them for it, they have a moral obligation to be trustworthy, and technically competent enough to guard

the users data with due diligence. The problem with most web software is designed is that by allowing the entire website to use the same credentials and have the same access from every point of the site, if any of them become vulnerable to some kind of SQL injection then all of their data is now unprotected and vulnerable. This is especially true for web software that stores credit card numbers to process in a batch mode, or passwords that use weak hash methods like an unsalted md5, which is the weakest hash because an hacker who's created a rainbow table can crack this in a matter of minutes, or plain-text passwords. The most important thing to remember is to use good web browsing practices like new, relatively strong, passwords for each sites. If it helps, you can also create a password for accounts that won't store sensitive information and you wouldn't be overly concerned with if someone else gained access to them compared to your other online accounts.

Privacy online has it's share of risks as well as the rewards and convenience of doing different types of business online. Although it's hard to protect your data and determine who should have it, you can educate yourself by looking for information to become more educated on the topic of phishing websites and stay away from them. The argument by many is that by having nothing to hide you shouldn't be afraid of disclosing all of the information you have and not securing it from the government. The practicality of that argument is disheartening though because even when computers can do many tasks with complete accuracy, and because to be err is to be human the computer can only act as an faithful servant to the instructions we've given to it. Similar to many examples in history, like the Salem witch trials at the end of the 15th century, it is an unchangeable truth that we are unable to determine who is guilty from who is innocent

and that taking someone's private information to convict them when they are innocent is violating the core value of what privacy means. Bruce Schneier, a world expert on security, said "Privacy protects us from abuses by those in power," which is true even when the abuses of power were only because we make mistakes.

Works Cited

- "Fair and Accurate Credit Transactions Act of 2003." 2003. US Congress. 7 Sept. 2008
<http://frwebgate.access.gpo.gov/cgi-bin/getdoc.cgi?dbname=108_cong_public_laws&docid=f:publ159.108>.
- "5.5.7 Using SSL for Secure Connections." MySQL :: MySQL 5.0 Reference Manual. 2008. 8 Sept. 2008 <<http://dev.mysql.com/doc/refman/5.0/en/secure-connections.html>>.
- Shneier, Bruce. "The Value of Privacy." Schneier on Security. 19 May 2006. 8 Sept. 2008
<http://www.schneier.com/blog/archives/2006/05/the_value_of_pr.html>.

Brett Bryant

Computer Architecture 330

Dr. Lin

March 31st, 2009

Ethics Paper: Data Thieves and Information Security

When purchasing products online or visiting a website that will handle your secure information there are many safeguards in place to try and ensure that the process goes along uninterrupted. Many malicious computer users try to find ways to take control over parts of that system to infiltrate your web of trust and keep your information for themselves. The increasing ability for even mediocre users to get their hands on exploits, or find ways in which to obtain stolen information, is an alarming trend that angers many who consider any tampering or stealing of information completely unethical. Since the internet is made up of many computers connected together to share information, there is no way to ever know if your information can be completely safe.

One of the most popular forms of identify theft comes from the use of malicious botnets. A "botnet" is a collection of host computers that have been compromised and infected with any arbitrary code that allows a host computer, or computers, controlled by a hacker to receive information about the host computer. Computers controlled by a botnet can send to their host computer any data they collect; for instance: bank credentials, passwords, or website browsing habits. The goal of stealing data is not the information they receive though, it's about being able to make money from your personal data.

Someone in possession of your stolen data can do a lot of harm to you. Some of the more

docile botnets are the ones that are designed to automatically click google ads to make the referrer money, however others are designed so that the zombie computers will take part in things like DDoS, brute force, and other malicious hacking methods. It's clear that it is unethical for anyb to have ability to control your information but yourself. Anybody attempting to make money from you by being part of a botnet is a thief.

The responsibility to try and keep your computer out of reach of a botnet is nobody's else's responsibility but their own. Although it would be nice if users could always expect to be in a world without threats, using your computer without following a few simple steps will lead to disaster. The first step is to run some kind of antivirus for your operating system that updates regularly. Without running any antivirus, viruses, trojan horses, and other malicious programs can infect your computer and allow it to be part of a botnet. The second step is to regularly update all of the software on your computer so that any security holes are patched as quickly as possible. The goal of an antivirus solution is to prevent already know viruses from exploiting already known bugs in software. The more regularly that you update your software the less known bugs will be available for hackers to exploit on your computer. The last step is to try and run as few services as possible open to the untrusted internet, and use a firewall to block the rest. Although your computer comes into contact with a lot of different things during use, there are a lot of viruses and malicious programs that spread through open services to the internet that aren't fire walled. It's a good idea to reduce this to as a few as possible to make yourself as safe as possible.

So even though it's possible for someone to steal your data, it's possible for you to protect yourself from them as well. This write-up doesn't mention other things you can do like avoiding fishing scams, etc, but information and training on those things are available online. You should

always protect yourself as much as possible from botnets so that the wrong people don't have too much control over the internet and the lives of honest people.

Presentations



Anagram Search (COIN 315 Data Structures)

The Anagram Search is a conceptual idea for using C++ to find any possible anagrams from a given sequence of characters. It works recursively to iterate through every combination among the letters given, and recursively again as it starts at the front of each random sequence of characters until looking into a hash table filled with dictionary words finds something with the given characters you are searching for. Once this process makes it through an entire string where every peice of it has a hash table entry with dictionary words then you can use those results to find all the anagrams.

Grade: 18 of 20



Bloggernaut (COIN 315 Data Structures)

Bloggernaut is a web application which allows users to sign up, create their own blog, and make their own posts using the Bloggernaut software. This presentation coincide with the finished project in the "Programs" section of the binder and was written in ASP.net, HTML, CSS, and SQL.

Grade: 85%

Anagram Searching

- Brett Bryant April 21st, 2008
- Creating The Dictionary
- Recursing Through The Dictionary

Creating The Dictionary pt.1

- Structure

```
hash_multimap < char *, char ** >
```

```
key: char *
```

```
value: char **
```

Creating The Dictionary pt.2

- Psuedo-Code

```
for each word from dictionary file:
  - let key = alphabetically sorted word
    [ bat = abt ]

  - insert word into hash_multimap....
    - lookup key in map
    - check for duplicates in hash entry
    - insert in new null-terminated value
      if key is not found or duplicate is
      at hash location, otherwise insert at
      the end
```

Creating The Dictionary pt.3

- Example hash_multimap entry:

```
dict["aet"] = { "ate", "eat", NULL };
```

Recurring Through The Dictionary

- Psuedo-Code:
 - Start with input string (of only alphabetic chars)
 - Recurse through each combination while making sure each substring is attempted to be at the beginning of an anagram
 - If an anagram is found with a predicate, add into hash_multimap that eliminates duplicates inside of that level of recursion
-

Recurring Through The Dictionary

- Return Value of Recursion: *char ****

- Example:

```
{ {"ate", "eat", "tea", NULL},  
  {"the", NULL}, NULL} =
```

- ate the, eat the, tea the

Bloggernaut (ASP.net)

- **Brett Bryant**
 - **November 16th, 2009**
- Dr. Lin**
Applied Networking

Bloggernaut (ASP.net)

- Overview
 - A "blog" system which consists of the ability for a user to create their own blogging environment by creating an account and making posts.

Bloggernaut (ASP.net)

■ Flowchart

